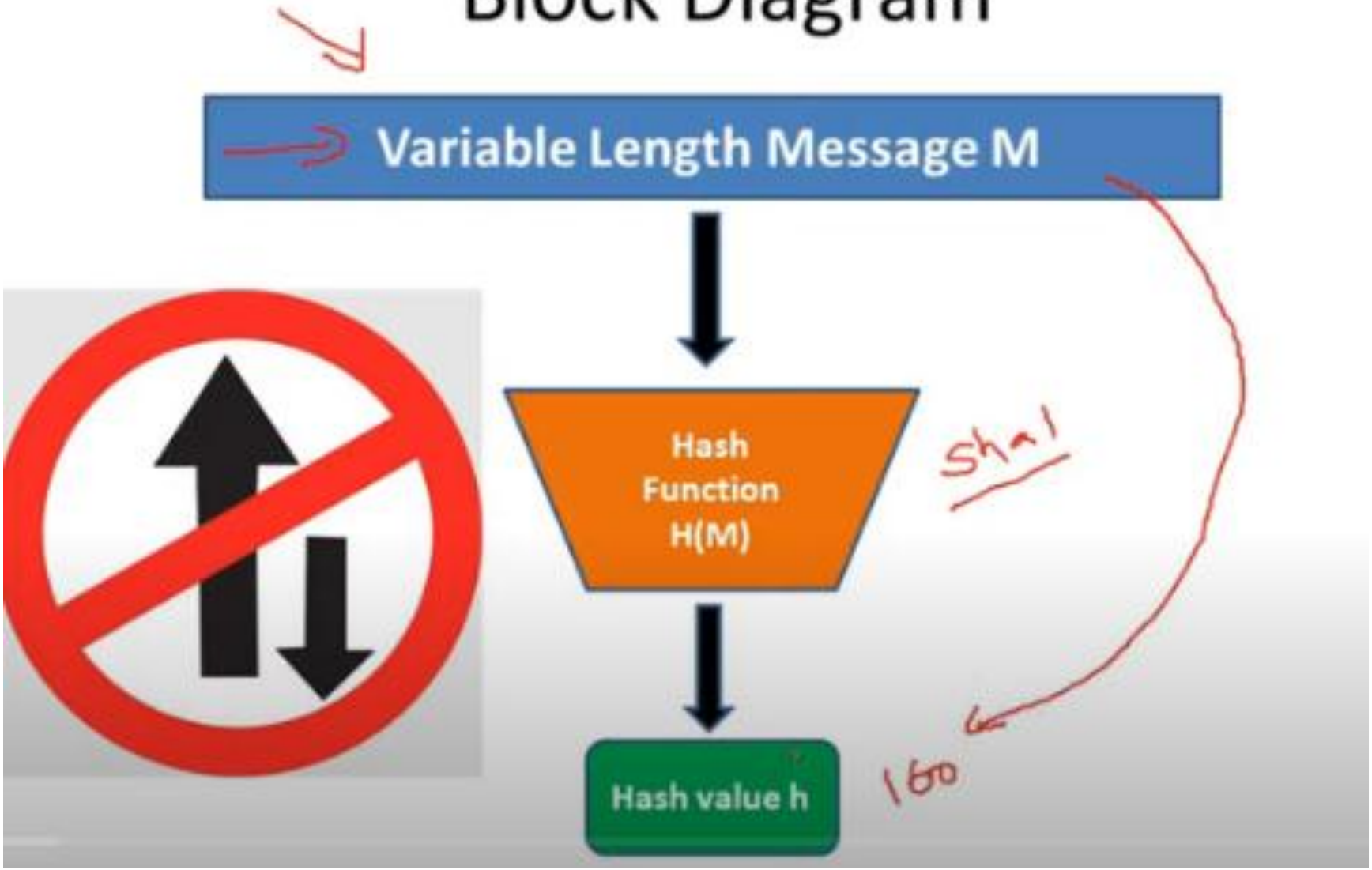
Hash Functions
(One Way Function)

What is a Hash Function ?

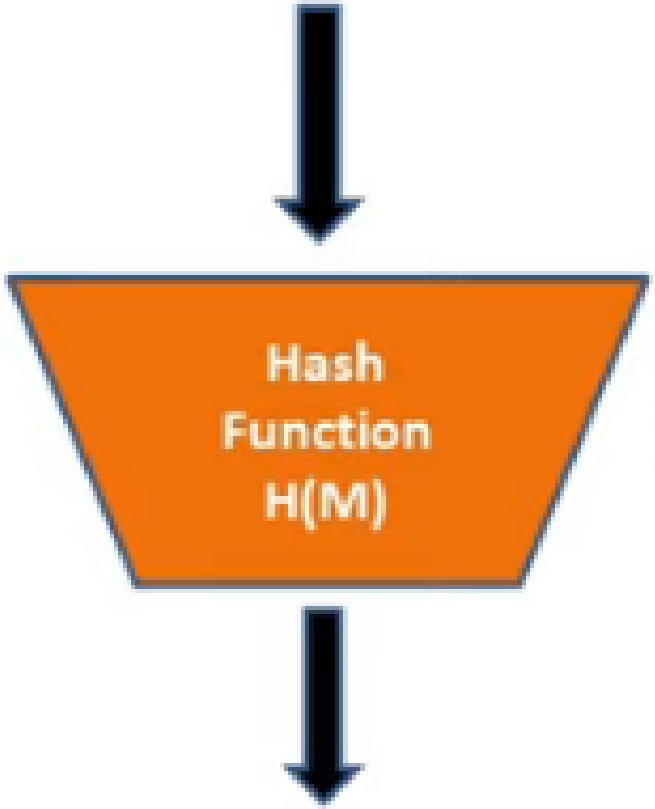
A hash function H accepts a variable-length block of data M as input and produces a fixed-size hash value .

$$h=H(M)$$

Block Diagram



krishnaraj



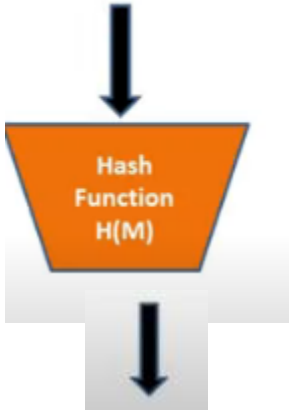
[a76506f37a163a3ded0c6e3f748315ee3e2511c6]

Properties of a Hash Function

- A "good" hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed and apparently random.
- A change to any bit or bits in results, with high probability, in a change to the hash code.

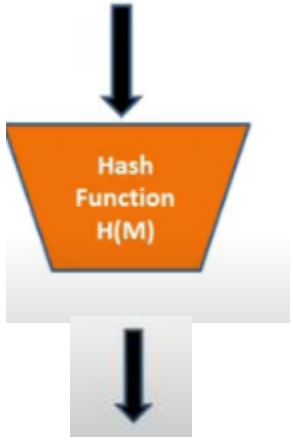
Avalanche Effect

- Krishnaraj



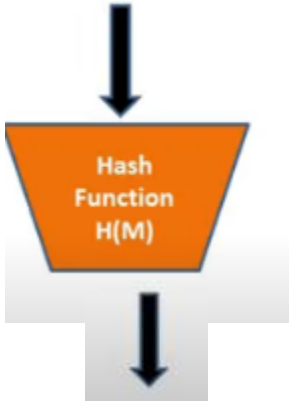
A76506f37a163a3d
Ed0c6e3f748315ee
3e2511c6

krishnara



ccc5552b99d7a70b
cf4a087a0675c0de0
dbc50bc

CNS



5bed37811e7609
Ce956a1012ee926
f8ba3

Should not have Collision

-
- krishnarai

CNS

A76506f37a163a3d
Ed0c6e3f748315ee
3e2511c6

A76506f37a163a3d
Ed0c6e3f748315ee
3e2511c6

Cryptographic Hash Functions

Hash functions used for Security Applications are known as Cryptographic Hash Functions

They have two important properties

It is computationally infeasible (because no attack is significantly more efficient than brute force) to find either

(a) a data object that maps to a pre-specified hash result (the one-way property) or

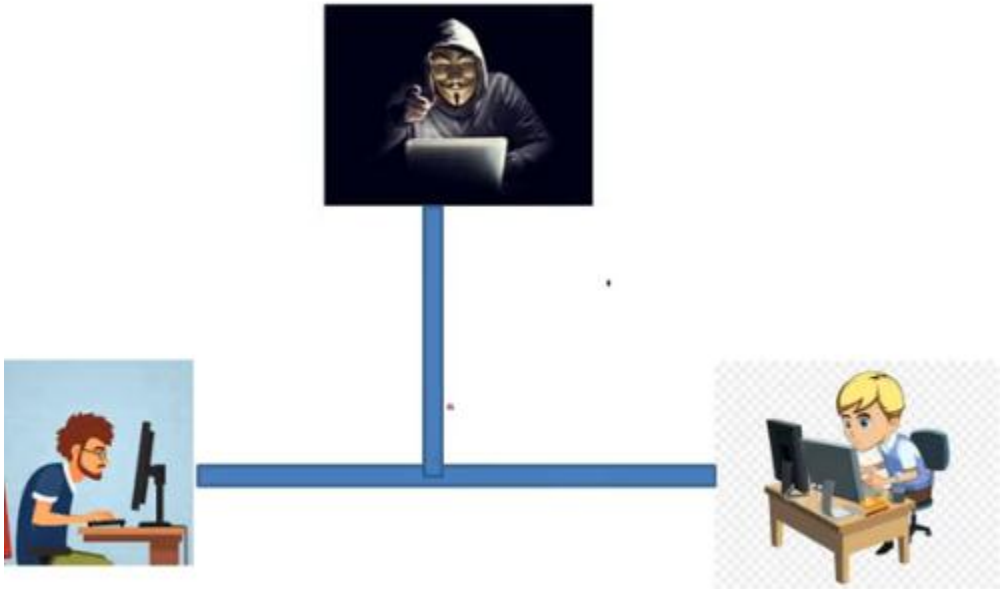
(b) two data objects that map to the same hash result (the collision-free property).

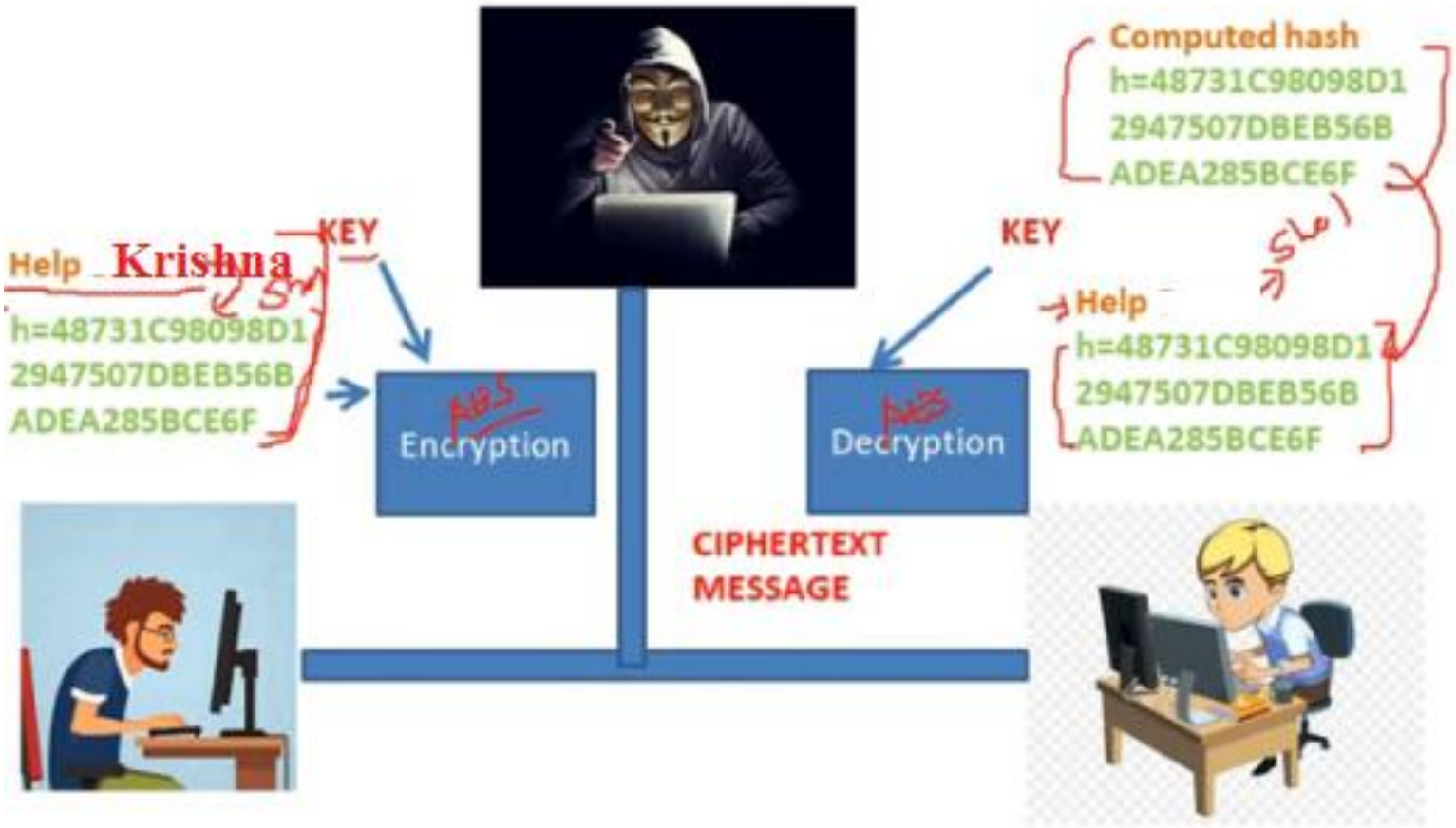
Application of Hash Functions in Cryptography

- Message Authentication
- Digital Signatures
- One Way Password File
- Intrusion Detection and Prevention

Message Authentication

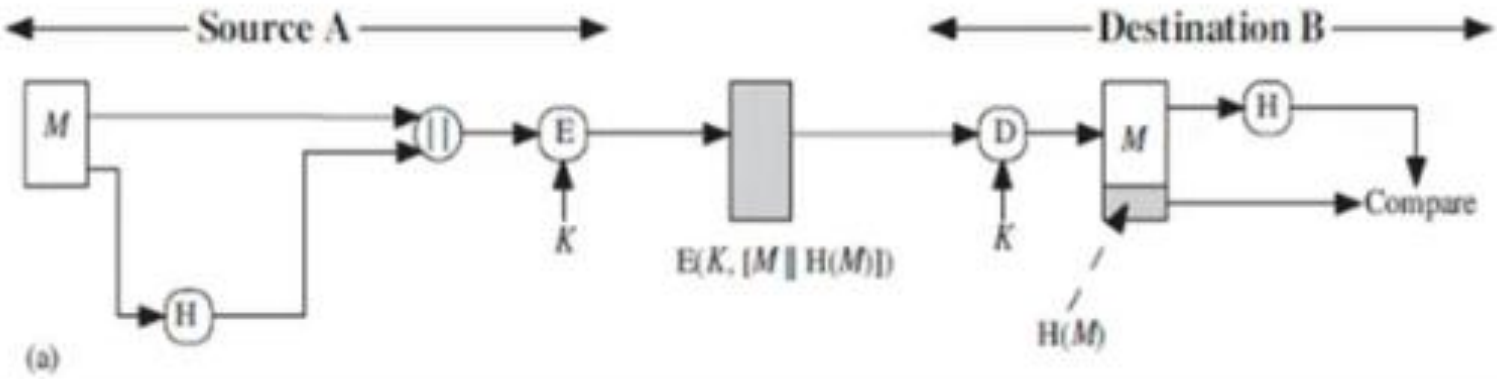
- Message authentication is a mechanism or service used to verify the integrity of a message.
- Message authentication assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay).


User A
User B

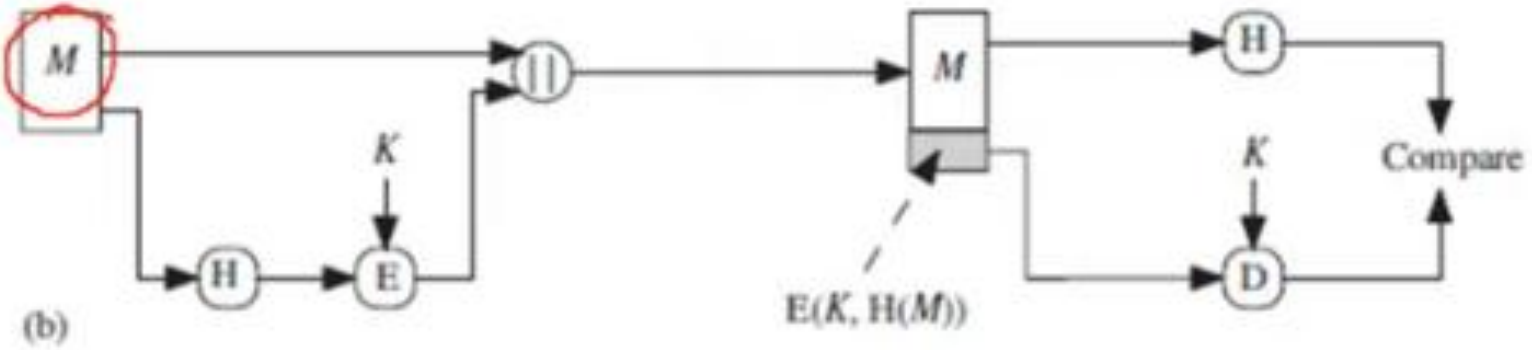


Method 1 –
Message and Hash Code Encrypted

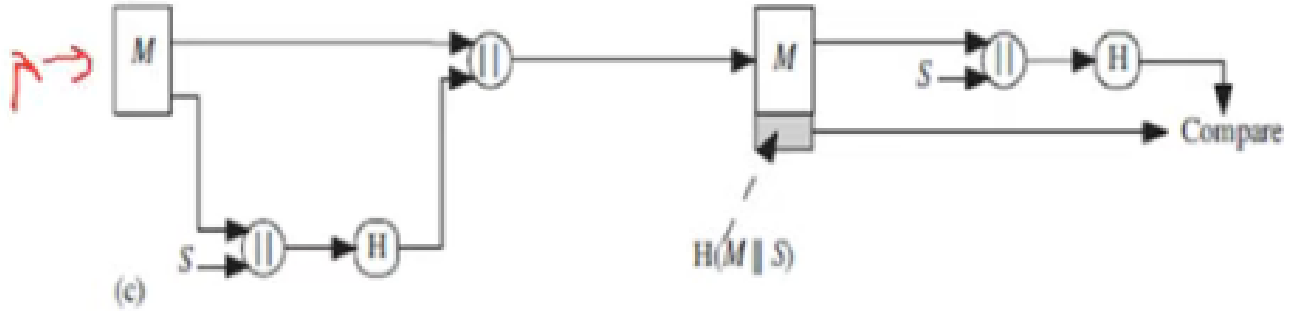
Method 1– Message and Hash Code Encrypted



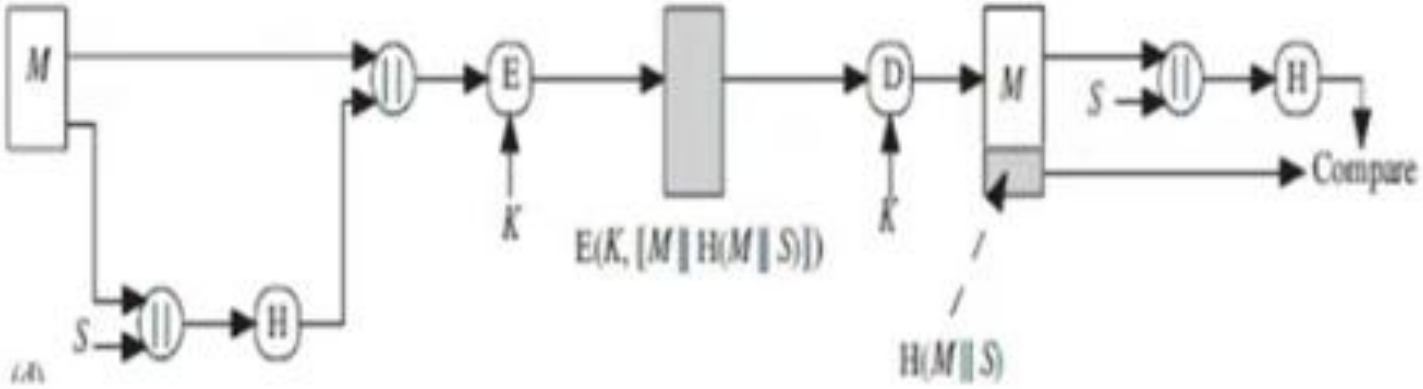
Method 2 - Only Hash Code is Encrypted



Method 3 - Sharing a Secret Value for Message Authentication



Method 4 - Adding Confidentiality to the previous method



Four Methods

Method 1

The message plus concatenated hash code is encrypted using symmetric encryption. Because only A and B share the secret key, the message must have come from A and has not been altered. The hash code provides the structure or redundancy required to achieve authentication. Because encryption is applied to the entire message plus hash code, confidentiality is also provided.

Method 2

Only the hash code is encrypted, using symmetric encryption. This reduces the processing burden for those applications that do not require confidentiality.

Four Methods

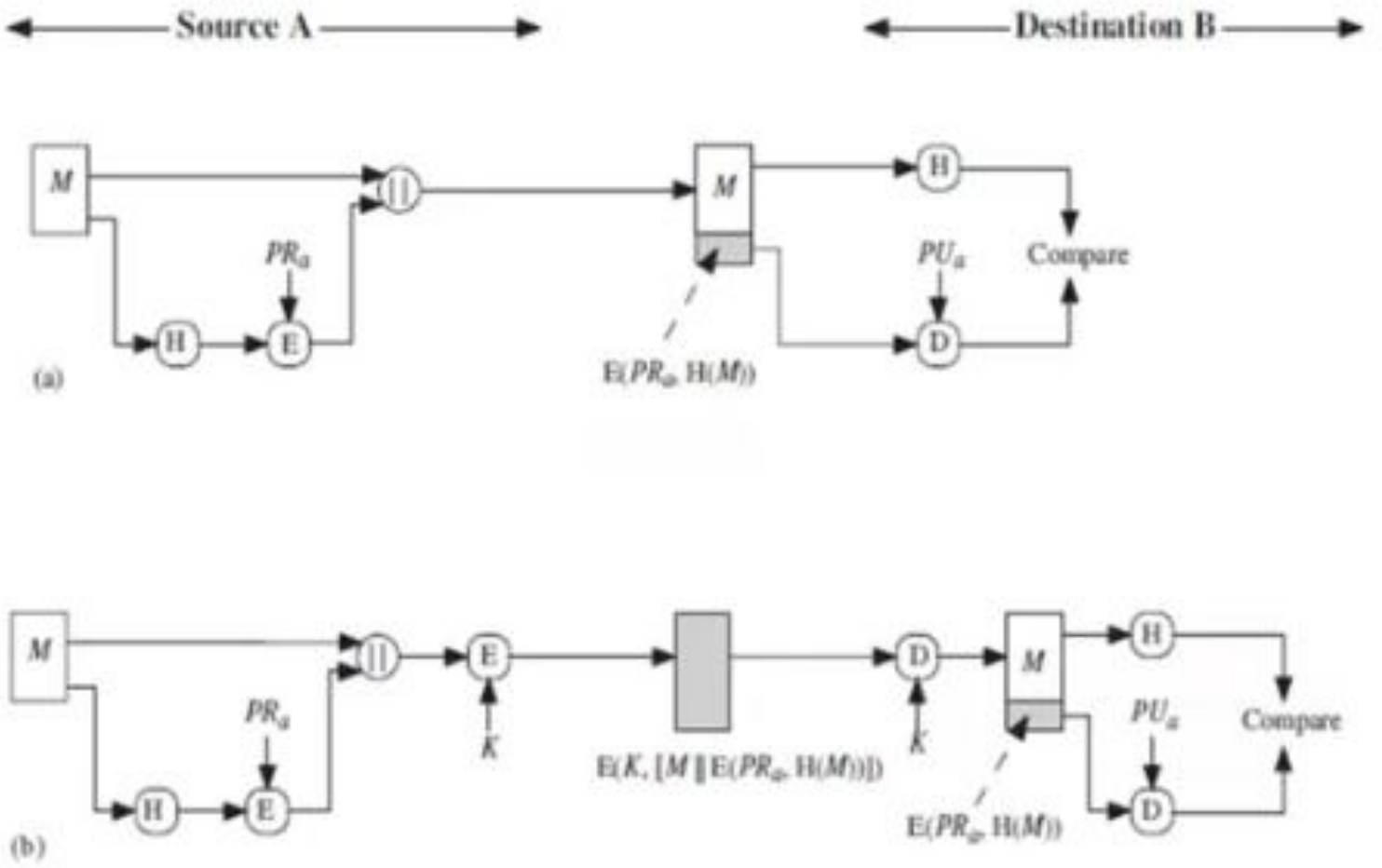
Method 3

It is possible to use a hash function but no encryption for message authentication. The technique assumes that the two communicating parties share a common secret value *S*. A computes the hash value over the concatenation of *M* and *S* and appends the resulting hash value to M. Because B possesses , it can re compute the hash value to verify. Because the secret value itself is not sent, an opponent cannot modify an intercepted message and cannot generate a false message.

Method 4

Confidentiality can be added to the approach of method (c) by encrypting the entire message plus the hash code.

Application of Hash Functions - Digital Signatures



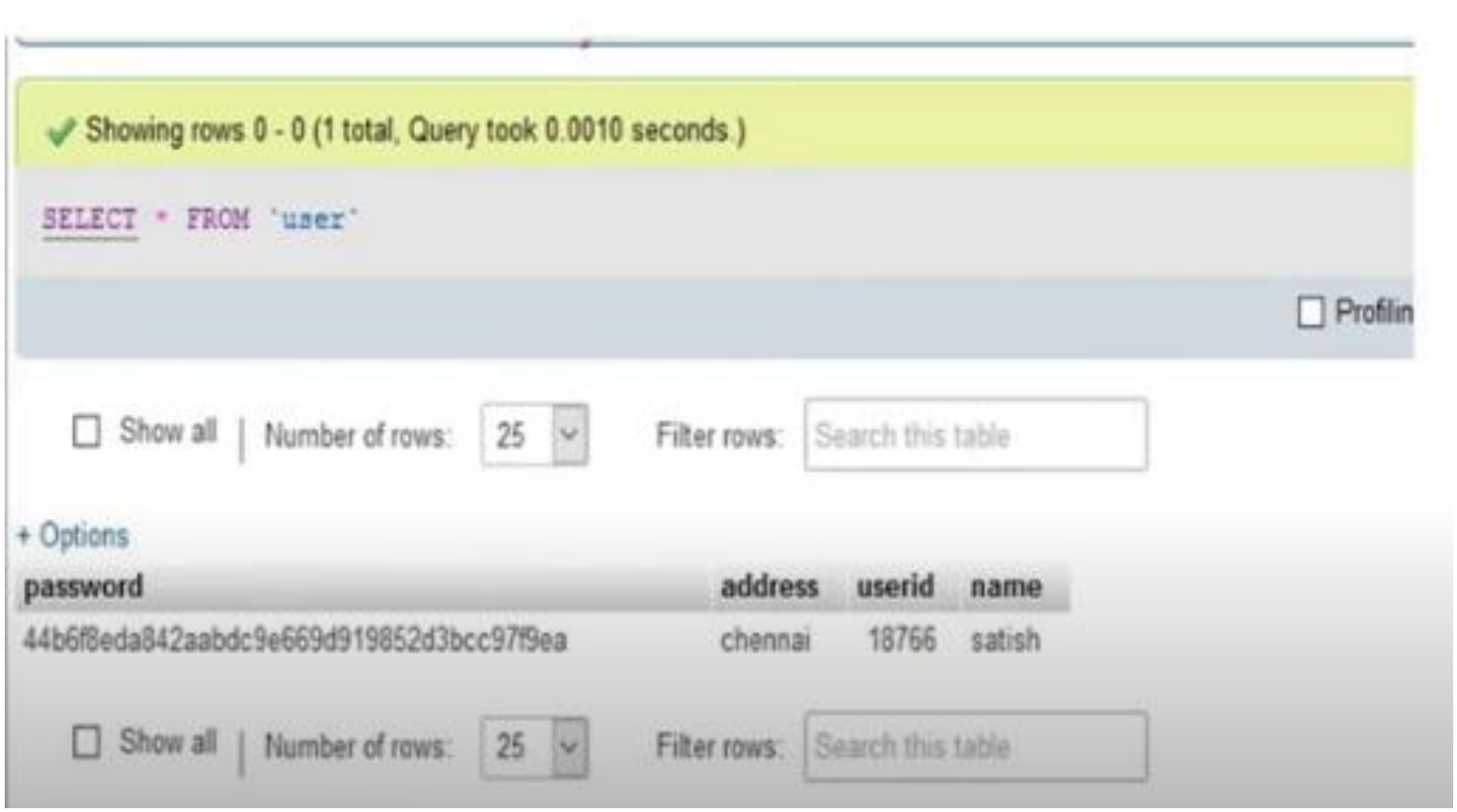
Application of Hash Functions - Digital Signatures

The hash code is encrypted, using public-key encryption with the sender's private key.

This provides authentication. It also provides a digital signature, because only the sender could have produced the encrypted hash code. In fact, this is the essence of the digital signature technique.

b. If confidentiality as well as a digital signature is desired, then the message plus the private-key-encrypted hash code can be encrypted using a symmetric secret key. This is a common technique.

One Time Password File



Intrusion and Virus Detection Systems

- Hash functions can be used for **intrusion detection** and **virus detection**.
- Store H(F) for each file on a system and secure the hash values
- One can later determine if a file has been modified by recomputing H(F).
- An intruder would need to change F without changing H(F).

Security Requirements for Cryptographic Hash Functions

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	$H(x)$ is relatively easy to compute for any given x, making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h, it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x, it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness.

Introduction to SHA

- The original specification of the algorithm was published in 1993 under the title *Secure Hash Standard (SHA-0)*, by U.S. government standards agency NIST (National Institute of Standards and Technology).
- It was withdrawn by the NSA shortly after publication and was superseded by the revised version, published in and commonly designated *SHA-1*
- **SHA-1** (**Secure Hash Algorithm 1**) is a cryptographic hash function which takes an input and produces a 160-bit hash value known as a message digest – typically rendered as a hexadecimal number, 40 digits long.

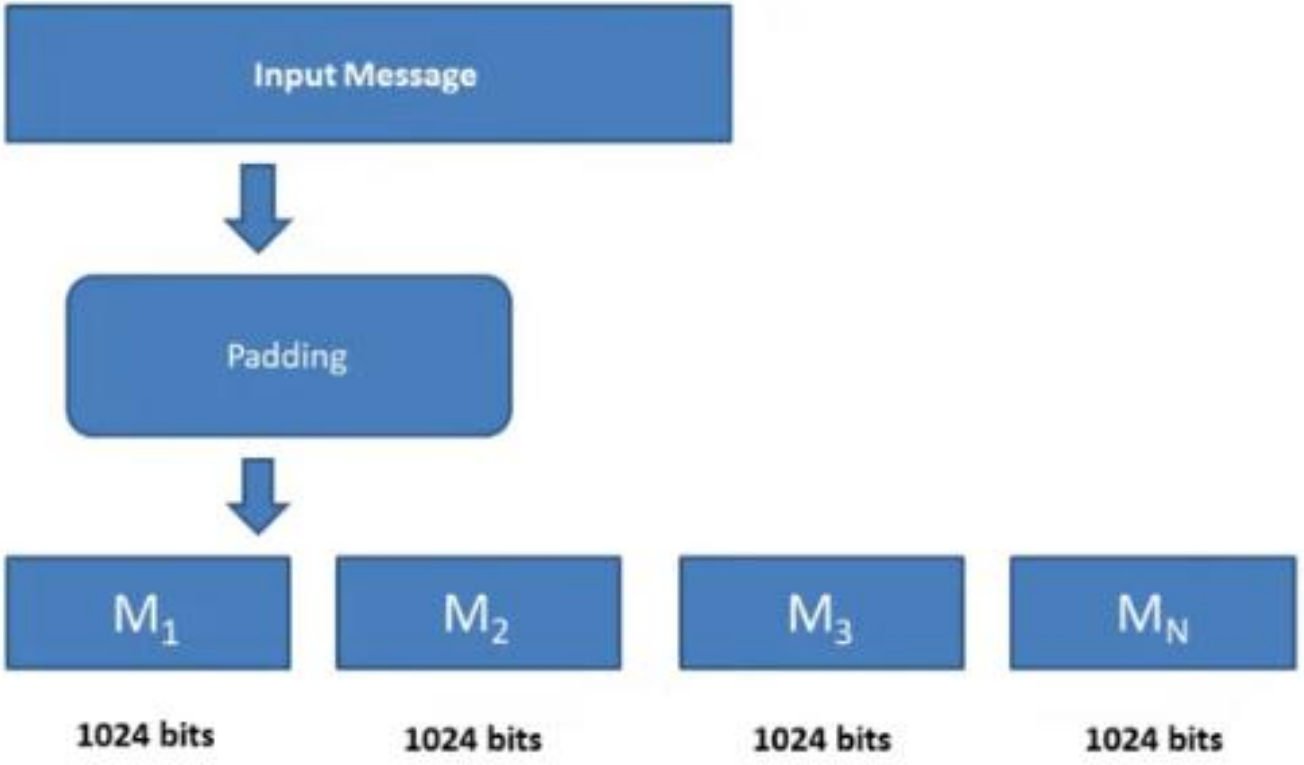
Secure Hash Algorithm (SHA512)

- Since 2005, SHA-1 has not been considered secure against well-funded opponents
- As of 2010 many organizations have recommended its replacement.
- NIST formally deprecated use of SHA-1 in 2011 and disallowed its use for digital signatures in 2013.
- The predecessor is SHA- 2. SHA-2 includes significant changes from its predecessor, SHA-1.
- The SHA-2 family consists of six hash functions with digests (hash values) that are 224, 256, 384 or 512 bits: SHA-224, SHA-256, SHA-384, **SHA-512**, SHA-512/224, SHA-512/256. SHA-256 and SHA-512 are novel hash functions computed with 32-bit and 64-bit words, respectively.

SHA 512 – Secure Hash Algorithm

Overview SHA 512

Input Message



Padding Example

- Consider Input Message "abc"

Represent in binary

01100001 01100010 01100011

Message_Length = 24 bits

Needed

Message_Length ≡896 mod 1024

Padding Example

- Consider Input Message "abc"

Represent in binary

01100001 01100010 01100011

Message_Length = 24 bits

Needed

Message_Length ≡896 mod 1024

Message_Length mod 1024 = 896

24 + 872 mod 1024 = 896

Pad 872 bits to message such that Message_Length mod 1024 = 896

872 bits to be padded – 1 bit followed by 871 zeros

871

$24 \equiv 896 \bmod 1024$

$24 \bmod 1024 = 896$

$24 \neq 896$

$24 + 872 \bmod 1024$

$896 \bmod 1024$

896

Padding Example

- Consider Input Message “abc”

Represent in binary

01100001 01100010 01100011

Message_Length = 24 bits

Needed

Message_Length ≡896 mod 1024

Message_Length mod 1024 = 896

6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000

Padding Example

896 + 128

6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000

Pad the original length of the message for 128 bits at the end

01100001 01100010 01100011

Message_Length = 24 bits

Hexadecimal value of 24 is 18

Message length represented in 128 bits in hexa decimal is

0000000000000000 0000000000000018

16 | 24 | 8

1

(18)

64

6162638000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000000
0000000000000000 0000000000000000 0000000000000000 0000000000000018

896 | 64 | 64 | 24

128

Exercise

- How many bits you will pad for input message length of 2348 bits?
- Message_Length ≡896 mod 1024

  2348 mod 1024 = 300

  2348 + 596 mod 1024 = 896

  Pad 596 bits where in 1 followed by 595 zeros

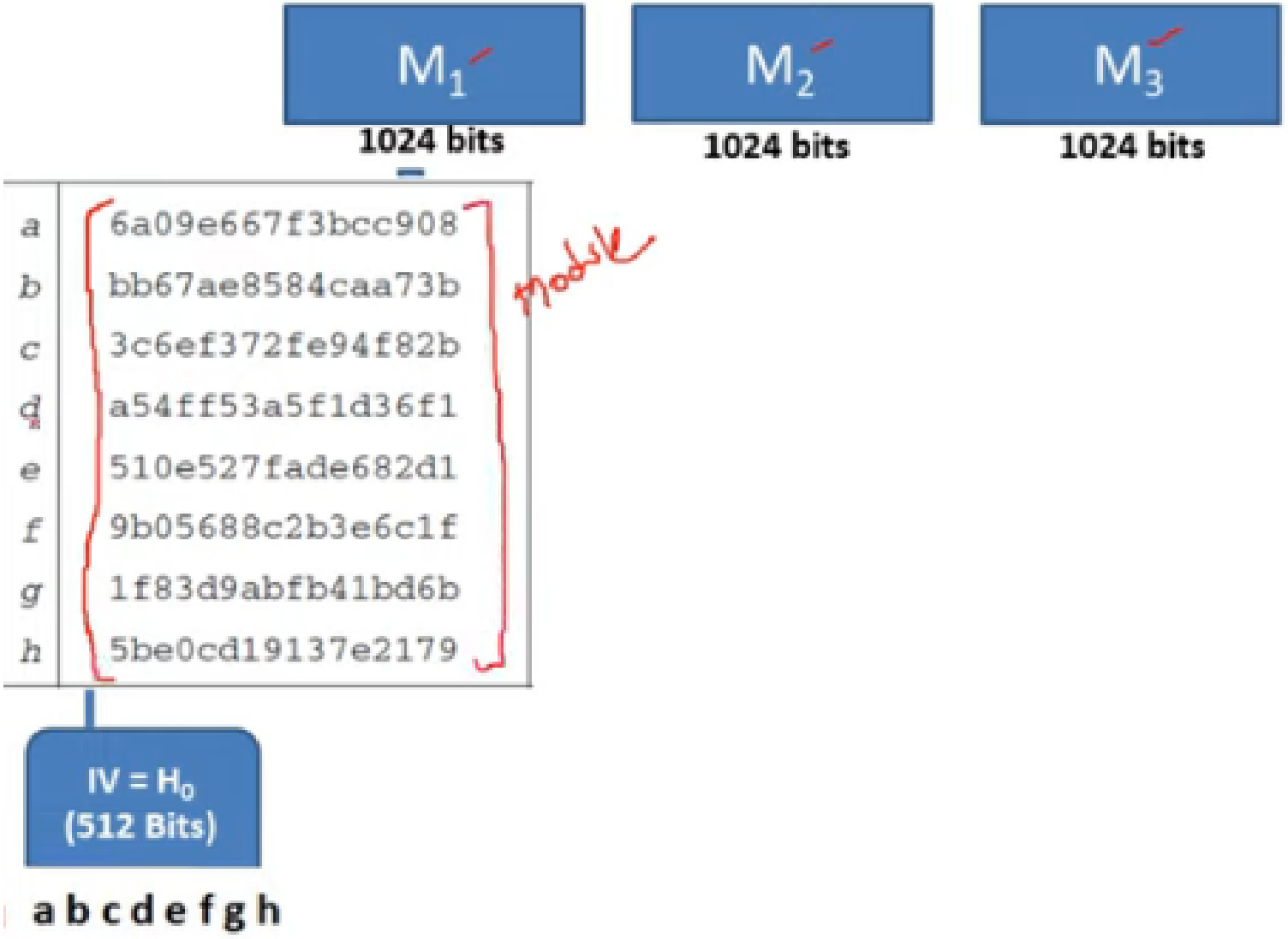
  Message length is 2944

  Add the message length (2348) as 128 bits at the end

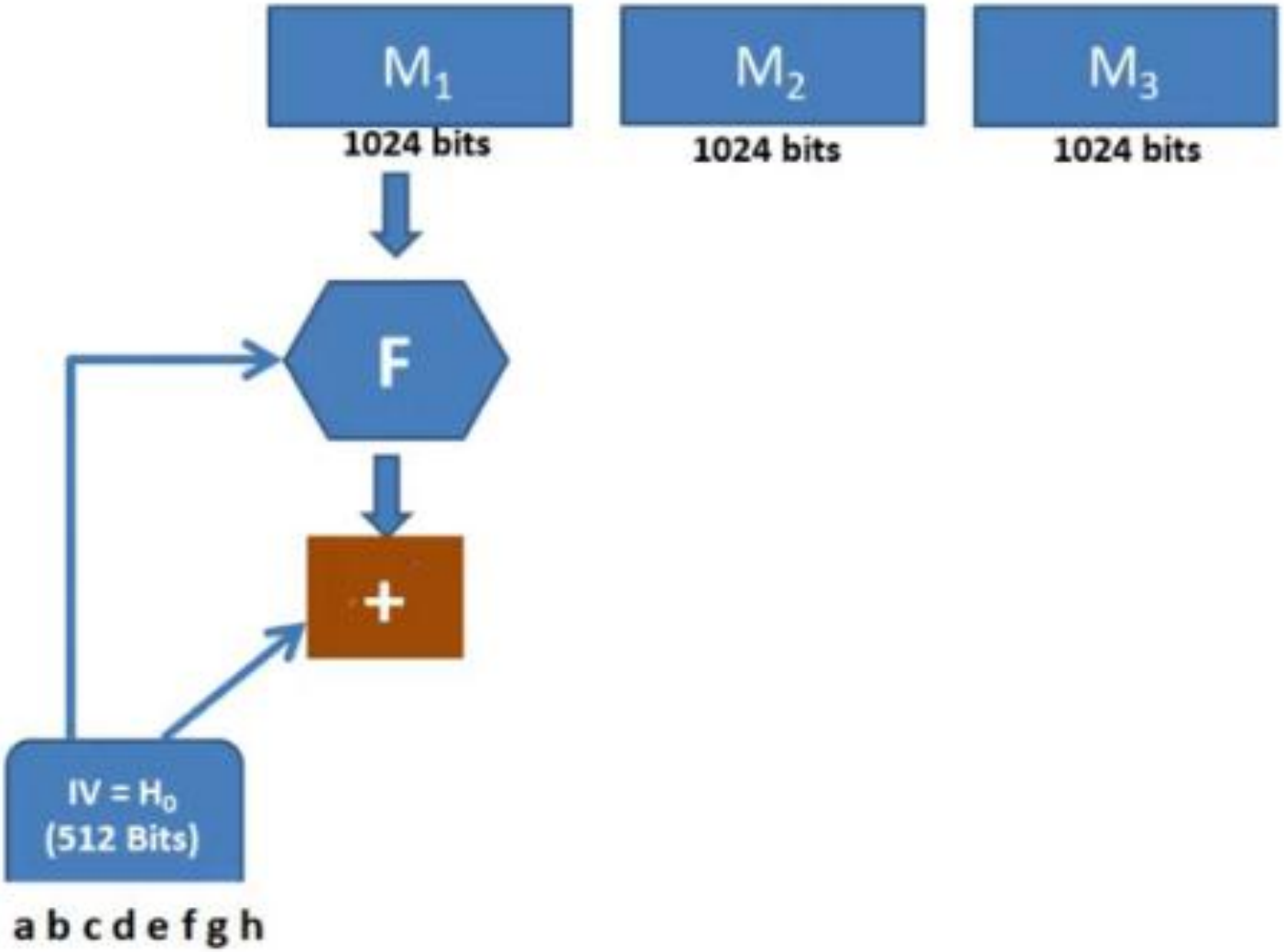
  Total bits will be 2944+ 128 = 3072

  This is nothing but Three M blocks of Size 1024 bits

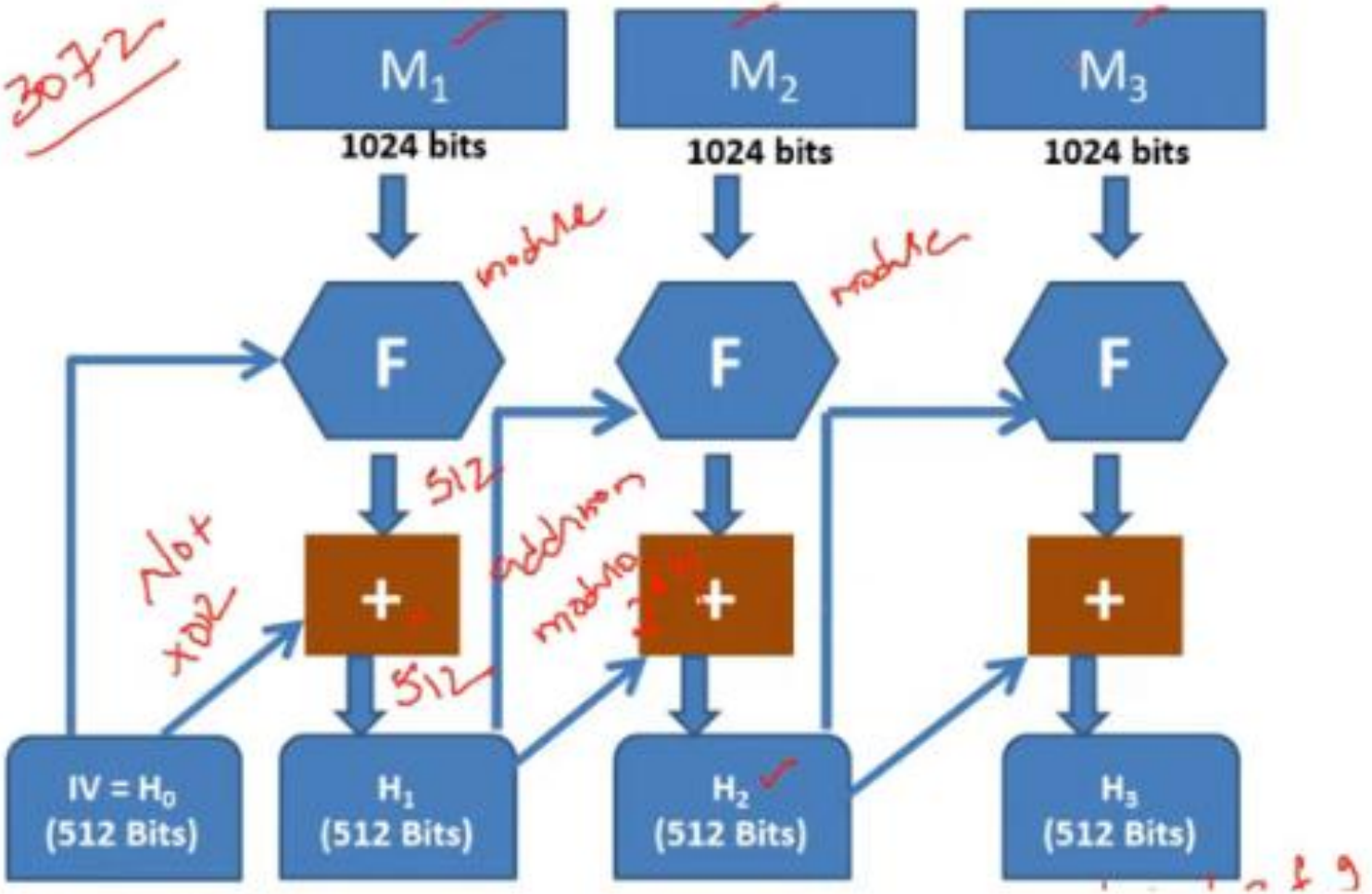
SHA 512 OVERALL BLOCK DIAGRAM



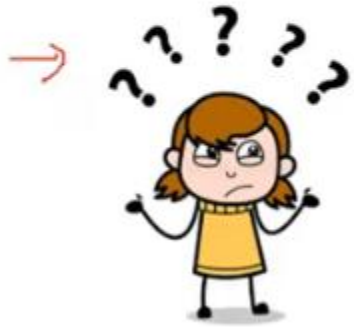
SHA 512 OVERALL BLOCK DIAGRAM



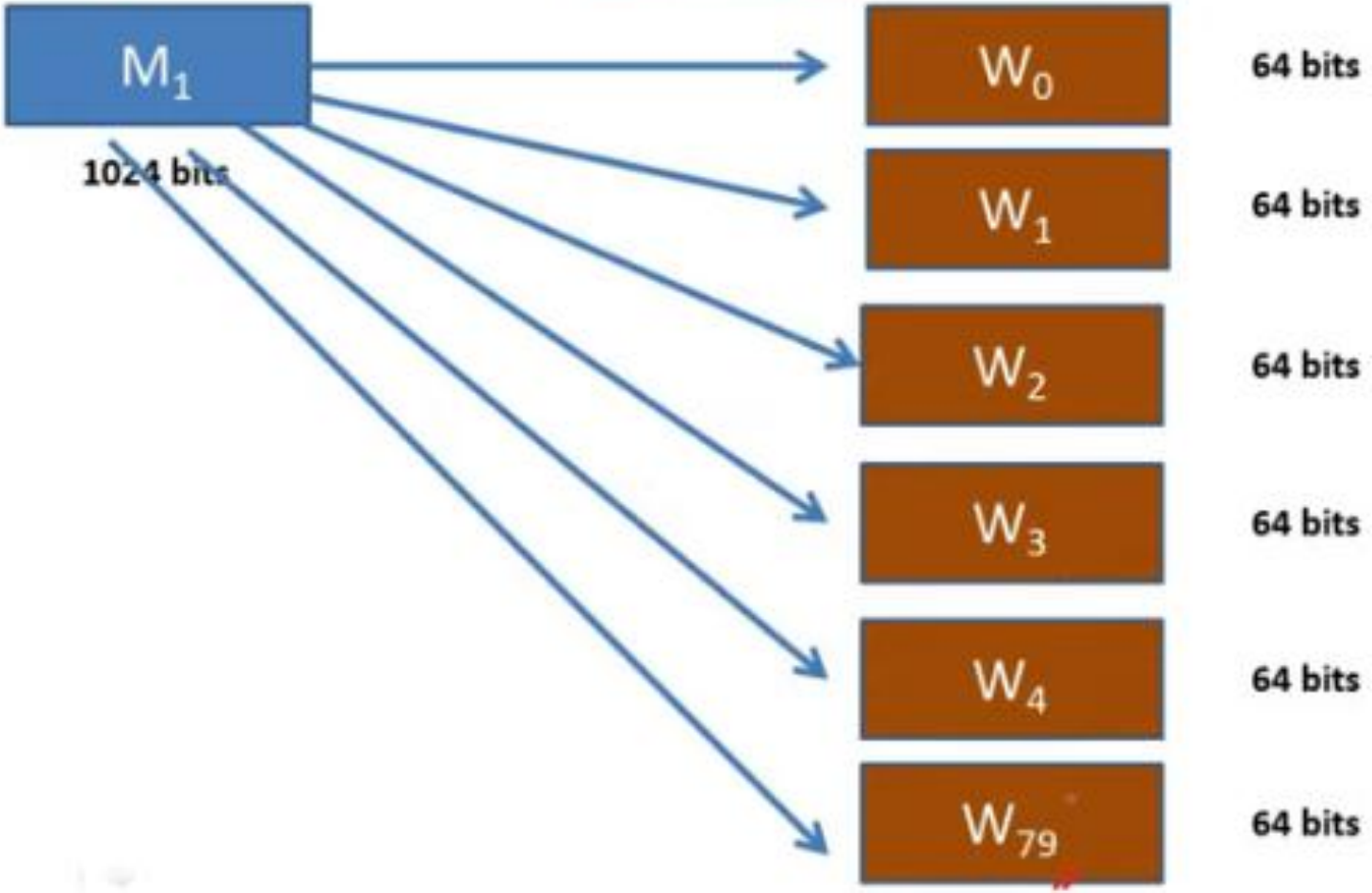
SHA 512 OVERALL BLOCK DIAGRAM



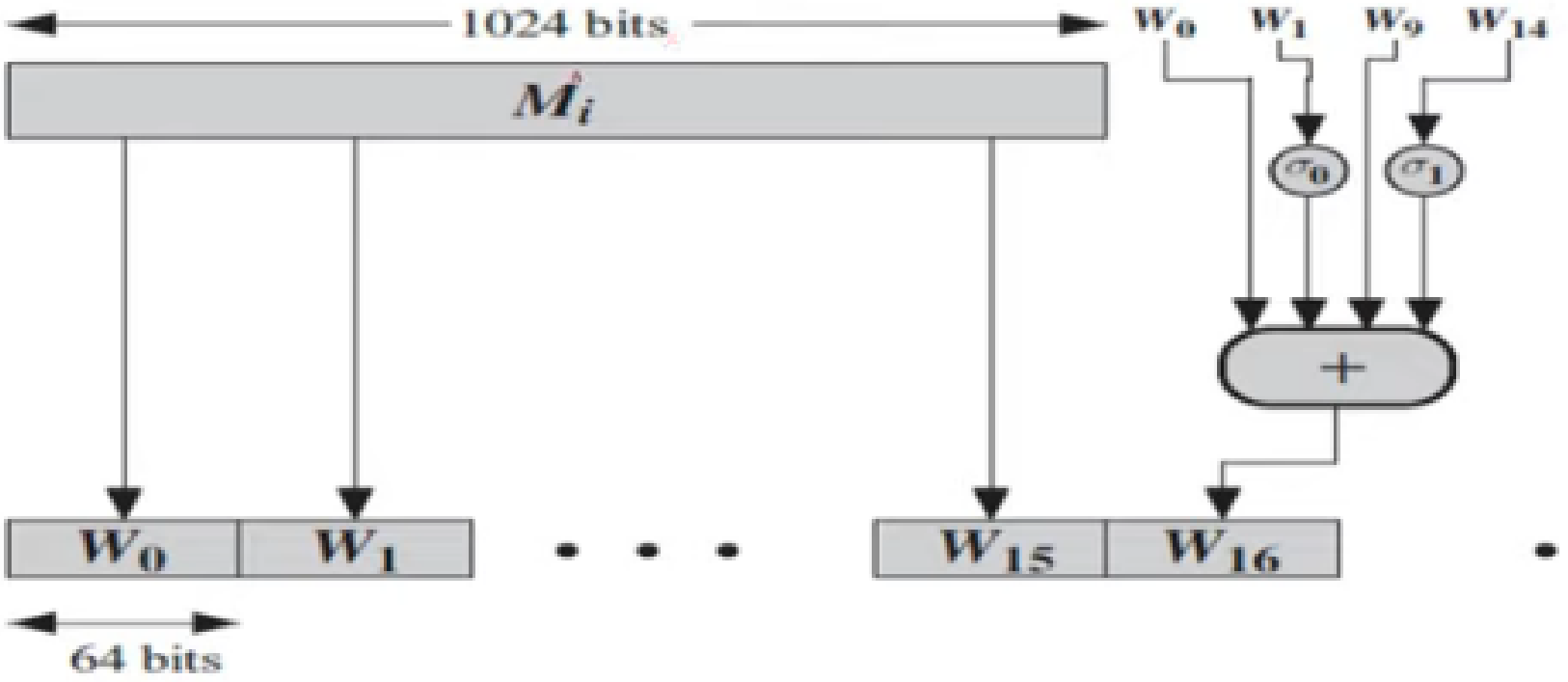
What is this
Module F ?



Module F – STEP 1
Message Scheduling



How to derive 80 words from a block of 1024 bits



$$W_t = \sigma_1^{512}(W_{t-2}) + W_{t-7} + \sigma_0^{512}(W_{t-15}) + W_{t-16}$$

where

$\sigma_0^{512}(x) = \text{ROTR}^1(x) \oplus \text{ROTR}^8(x) \oplus \text{SHR}^7(x)$

$\sigma_1^{512}(x) = \text{ROTR}^{19}(x) \oplus \text{ROTR}^{61}(x) \oplus \text{SHR}^6(x)$

$\text{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

$\text{SHR}^n(x)$ = left shift of the 64-bit argument x by n bits with padding by zeros on the right

$+$ = addition modulo 2^{64}